Perplexity Claude Style SaaS Project Blueprint

Perplexity/Claude Style SaaS Project Blueprint
1. Product Goal
2. Current Status Summary
   2.1 Frontend Current Status
   2.2 Backend Current Status
3. Backend Architecture
   3.1 Backend Target Folder Structure
   3.2 Backend Core Request Workflow
   3.3 Search/Answer Engine Workflow
   3.4 Backend Diagrams
4. Backend Files Explained
   4.1 `src/app.js`
   4.2 `src/controllers/search.controller.js`
   4.3 `src/services/search.service.js`
   4.4 `src/services/scraper.service.js`
   4.5 `src/services/groq.service.js`
   4.6 `src/services/citation.service.js`
   4.7 `src/services/stream.service.js`
   4.8 `src/controllers/chat.controller.js`
   4.9 `src/models/source.model.js`
   4.10 `src/models/usage.model.js`
   4.11 `src/middlewares/rateLimit.middleware.js`
   4.12 `src/services/cache.service.js`
5. Backend Features
   5.1 Auth Feature
   5.2 Search Feature
   5.3 Streaming Feature
   5.4 Chat History Feature
   5.5 Library Feature
   5.6 Settings Feature
   5.7 Billing Feature
   5.8 File Upload Feature
   5.9 Spaces Feature
   5.10 Public Sharing Feature
6. Backend API Plan
   6.1 Auth APIs
   6.2 Search APIs
   6.3 Chat APIs
   6.4 Library APIs
   6.5 Settings/User APIs
   6.6 Billing APIs

- 6.7 File APIs
- 6.8 Spaces APIs

7. APIs And External Services Needed
   - 7.1 LLM APIs
   - 7.2 Search APIs
   - 7.3 Scraping APIs
   - 7.4 Database
   - 7.5 Cache/Rate Limit
   - 7.6 Email
   - 7.7 Billing
   - 7.8 File Storage
8. Frontend Architecture
   - 8.1 Frontend Target Folder Structure
   - 8.2 Frontend App Workflow
   - 8.3 Search/Chat Frontend Workflow
   - 8.4 Frontend Diagram
   - 8.5 Frontend Files Explained
9. Frontend Design And UI/UX
   - 9.1 Design Direction
   - 9.2 Color Theory
   - 9.3 Typography
   - 9.4 Layout Rules
   - 9.5 Home Page Design
   - 9.6 Chat Thread Design
   - 9.7 Source Panel Design
   - 9.8 Loading States
   - 9.9 Error States
   - 9.10 Accessibility
10. Frontend Features
    - 10.1 Auth
    - 10.2 Chat/Search
    - 10.3 Library
    - 10.4 Settings
    - 10.5 Billing
    - 10.6 Files
    - 10.7 Spaces
    - 10.8 Share
11. What To Build First
    - Phase 1: Core Answer Engine
    - Phase 2: Streaming
    - Phase 3: Chat History
    - Phase 4: SaaS Limits
    - Phase 5: Advanced Features
12. Done vs Remaining
    - 12.1 Done
    - 12.2 Remaining
13. Environment Variables Needed
14. Final Recommended Architecture

Perplexity/Claude Style SaaS Project Blueprint

Prepared for this workspace:

```
C:\Users\singh\Desktop\Coding_Files\WEB_DEVELOPMENT\Cohort-2.0\Backend\lec-32
         to lec-39 (Perplexity)
```

This document explains the full project from backend to frontend: what already exists, what remains, what folders/files should exist, what each file is for, which APIs are needed, how the core features work, and how the UI should be designed.

1. Product Goal

The product should behave like a real SaaS AI app inspired by Perplexity and Claude.

It should not be only a normal chatbot. The important product idea is:

```
Question -> live/searchable knowledge -> source extraction -> AI synthesis ->
         cited answer -> saved conversation
```

The app should eventually support:

- User accounts.
- Login/register/logout.
- Email verification and password reset.
- AI chat.
- Live web search.
- Source scraping.
- Citation-backed answers.
- Streaming responses.
- Chat history.
- Library/history management.
- User settings.
- Usage limits.
- Billing/subscriptions.
- File upload and document chat.
- Spaces/workspaces.
- Shareable public answer pages.

2. Current Status Summary

2.1 Frontend Current Status

Your frontend already has a React/Vite project and a feature-based structure.

Already created:

```
Frontend/src/
  main.jsx

  app/
    App.jsx
    app.routes.jsx
    app.store.js
    index.css

  components/
    layout/
      AppLayout.jsx
      Sidebar.jsx
      Topbar.jsx
      UserMenu.jsx

    ui/
      Button.jsx
      Dropdown.jsx
      EmptyState.jsx
      Input.jsx
      Modal.jsx
      Tabs.jsx

  features/
    auth/
      auth.slice.js
      components/Protected.jsx
      hook/useAuth.js
      pages/Login.jsx
      pages/Register.jsx
      service/auth.api.js

    chat/
      chat.slice.js
      components/ChatHeader.jsx
      components/ChatInput.jsx
      components/ChatSidebar.jsx
      components/MessageBubble.jsx
      components/ModelSelector.jsx
      components/PromptCard.jsx
      components/SourcePanel.jsx
      components/SuggestedPrompts.jsx
      components/WebToggle.jsx
      hook/useChat.js
      pages/ChatThread.jsx
      pages/Dashboard.jsx
      pages/Home.jsx
      services/chat.api.js

    library/
```

```
      library.slice.js
      components/ChatListItem.jsx
      components/LibraryFilters.jsx
      components/LibrarySearch.jsx
      components/LibrarySidebar.jsx
      hook/useLibrary.js
      pages/Library.jsx
      service/library.api.js

    settings/
      settings.slice.js
      components/AccountPlan.jsx
      components/DangerZone.jsx
      components/ProfileSettings.jsx
      components/SettingsSidebar.jsx
      hook/useSettings.js
      pages/Settings.jsx
      service/settings.api.js
```

Implemented well:

- Auth pages have real code.
- Protected route exists.
- Redux store exists.
- Global CSS exists.
- Login/Register UI exists.

Created but mostly empty:

- Shared layout files.
- Shared UI files.
- Chat components.
- Chat Home and ChatThread pages.
- Library feature.
- Settings feature.

Important naming fix:

`features/chat/services/chat.api.js`

should become:

`features/chat/service/chat.api.js`

because `auth`, `library`, and `settings` use `service`, singular.

2.2 Backend Current Status

Your backend already has Express, MongoDB, auth, middleware, and basic models.

Already created:

```
Backend/
  server.js
  package.json
  .env

  src/
    app.js

    config/
      database.js

    controllers/
      auth.controller.js

    middlewares/
      auth.middleware.js
      error.middleware.js
      validator.middleware.js

    models/
      user.model.js
      chat.model.js
      message.model.js

    routes/
      auth.routes.js

    services/
      ai.services.js
      mail.service.js

    utils/
      ApiError.js
      asyncHandler.js

    validators/
      auth.validator.js
```

Implemented well:

- Express app setup.
- MongoDB connection.
- Auth routes.
- Auth controller.
- JWT middleware.
- Error handler.
- Validator middleware.
- User model.
- Chat model.
- Message model.
- Mail service.
- Basic AI service.

Still missing:

- Search route.
- Chat routes.
- Source model.
- Search controller.
- Chat controller.
- Tavily/Brave/Exa search service.
- Jina/Firecrawl scraper service.
- Groq/OpenAI/Anthropic LLM service.
- SSE streaming.
- Citation parsing and validation.
- Usage limits.
- Rate limiting.
- Billing/subscriptions.
- File upload.
- Spaces/workspaces.
- Public sharing.

3. Backend Architecture

3.1 Backend Target Folder Structure

For a real SaaS-style backend, target this structure:

```
Backend/src/
  app.js

  config/
    database.js
    env.js
    cors.js
    multer.js
    redis.js

  constants/
    models.js
    plans.js
    roles.js
    searchModes.js

  controllers/
    auth.controller.js
    search.controller.js
    chat.controller.js
    user.controller.js
    library.controller.js
    settings.controller.js
    billing.controller.js
    file.controller.js
    space.controller.js
```

```
    share.controller.js

  lib/
    groqClient.js
    openaiClient.js
    anthropicClient.js
    tavilyClient.js
    braveClient.js
    exaClient.js
    redisClient.js
    stripeClient.js

  middlewares/
    auth.middleware.js
    error.middleware.js
    validator.middleware.js
    rateLimit.middleware.js
    subscription.middleware.js
    upload.middleware.js
    role.middleware.js

  models/
    user.model.js
    chat.model.js
    message.model.js
    source.model.js
    search.model.js
    usage.model.js
    subscription.model.js
    file.model.js
    space.model.js
    sharedPage.model.js
    apiKey.model.js

  routes/
    auth.routes.js
    search.routes.js
    chat.routes.js
    user.routes.js
    library.routes.js
    settings.routes.js
    billing.routes.js
    file.routes.js
    space.routes.js
    share.routes.js

  services/
    ai.services.js
    groq.service.js
    search.service.js
    scraper.service.js
```

```
    citation.service.js
    stream.service.js
    queryRewrite.service.js
    chat.service.js
    user.service.js
    billing.service.js
    file.service.js
    space.service.js
    cache.service.js
    usage.service.js

  validators/
    auth.validator.js
    search.validator.js
    chat.validator.js
    user.validator.js
    settings.validator.js
    billing.validator.js
    file.validator.js
    space.validator.js

  utils/
    ApiError.js
    ApiResponse.js
    asyncHandler.js
    buildPrompt.js
    normalizeUrl.js
    timeout.js

  jobs/
    usageReset.job.js
    cleanup.job.js
    email.job.js

  queues/
    mail.queue.js
    search.queue.js
```

You do not need to implement all files at the same time. This is the target structure for a serious SaaS application.

3.2 Backend Core Request Workflow

This is the most important project workflow.

```
Frontend
  |
  | POST /api/search
  | body: { query, mode, model, chatId }
  v
search.routes.js
```

```
  |
  v
search.controller.js
  |
  +-> validate query
  +-> check auth/user
  +-> check usage limit
  +-> optional query rewrite
  +-> call search.service.js
  +-> call scraper.service.js
  +-> call citation.service.js
  +-> call groq.service.js
  +-> stream response with stream.service.js
  +-> save chat/message/source data with chat.service.js
  |
  v
Frontend renders streaming answer and sources
```

3.3 Search/Answer Engine Workflow

```
User asks:
"What is RAG in AI?"

Step 1: Validate
  query exists
  query length is safe
  user has remaining searches

Step 2: Query rewrite
  "RAG in AI" becomes:
  "retrieval augmented generation in artificial intelligence explanation
         sources"

Step 3: Search
  Tavily/Brave/Exa returns URLs, titles, snippets, scores

Step 4: Scrape/extract
  Jina Reader extracts readable markdown from top URLs

Step 5: Context assembly
  source 1 content
  source 2 content
  source 3 content
  trimmed to safe token limit

Step 6: LLM generation
  prompt tells model:
  "Answer only from sources. Cite claims like [1], [2]."

Step 7: Streaming
  model tokens stream to frontend using SSE
```

```
Step 8: Save
  MongoDB stores chat, user message, AI message, sources

Step 9: Render
  Frontend shows answer, citations, source cards, follow-up input
```

3.4 Backend Diagrams

High-level backend:

```
                      +------------------+
                      |     Frontend     |
                      +---------+--------+
                                |
                                v
                        +-------+-------+
                        | Express app   |
                        +-------+-------+
                                |
        +-----------------------+-----------------------+
        |                       |                       |
        v                       v                       v
  +-----+------+          +------+-------+          +-----+------+
  | Auth APIs  |          | Search APIs  |          | Chat APIs  |
  +-----+------+          +------+-------+          +-----+------+
        |                       |                       |
        v                       v                       v
  +-----+------+    +------------+-------------+    +-----+------+
  | User model |    | Search/Scrape/LLM stack |    | Chat model |
  +------------+    +------------+-------------+    +------------+
                                |
           +--------------------+--------------------+
           |                    |                    |
           v                    v                    v
       Tavily/Brave          Jina Reader          Groq/OpenAI
```

SSE streaming:

```
Groq stream
  |
  | token chunk
  v
Express response
  |
  | res.write("data: {...}\n\n")
  v
Browser EventSource/fetch stream
  |
  v
React state appends token
```

```
  |
  v
StreamingAnswer component updates live
```

Citation flow:

```
Search results
  |
  v
Number sources
  [1] arxiv.org
  [2] langchain.com
  [3] huggingface.co
  |
  v
Prompt model with numbered sources
  |
  v
Model returns:
  "RAG combines retrieval with generation [1][2]."
  |
  v
Frontend parser finds [1][2]
  |
  v
CitationBadge links to SourceCard
```

4. Backend Files Explained

4.1 `src/app.js`

Purpose:

- Creates Express app.
- Adds middleware.
- Registers routes.
- Adds global 404 handler.
- Adds error handler.

Current routes:

```
app.use("/api/auth", userRouter);
```

Needed routes later:

```
app.use("/api/search", searchRouter);
app.use("/api/chats", chatRouter);
app.use("/api/users", userRouter);
app.use("/api/settings", settingsRouter);
app.use("/api/billing", billingRouter);
app.use("/api/files", fileRouter);
```

```
app.use("/api/spaces", spaceRouter);
app.use("/api/share", shareRouter);
```

4.2 `src/controllers/search.controller.js`

Purpose:

- Handles `POST /api/search`.
- Reads `query`, `mode`, `model`, and optional `chatId`.
- Calls services in correct order.
- Sends normal JSON or streams SSE.

What logic it needs:

- Validate request.
- Check user auth.
- Check plan/usage limits.
- Call query rewrite service.
- Call search service.
- Call scraper service.
- Build citations.
- Call LLM service.
- Save chat and messages.
- Return answer and sources.

Main endpoints:

```
POST /api/search
POST /api/search/stream
```

4.3 `src/services/search.service.js`

Purpose:

- Search the web.
- Return structured source candidates.

Possible APIs:

- Tavily: best for AI answer apps.
- Brave Search: good independent search.
- Exa: good semantic/research search.

Expected output:

```
[
  {
    title: "What is Retrieval Augmented Generation?",
    url: "https://example.com/rag",
    content: "Snippet...",
    score: 0.92,
    domain: "example.com"
```

```
  }
]
```

4.4 `src/services/scraper.service.js`

Purpose:

- Extract readable content from URLs.

Recommended APIs:

- Jina Reader: `https://r.jina.ai/http://...`
- Firecrawl: stronger fallback for difficult sites.

Workflow:

```
top search URLs
  |
  v
Promise.all with timeout
  |
  +-> Jina Reader URL 1
  +-> Jina Reader URL 2
  +-> Jina Reader URL 3
  |
  v
clean markdown/text
```

Important:

- Always set timeout.
- Do not scrape too many URLs.
- Use snippets as fallback if scraping fails.

4.5 `src/services/groq.service.js`

Purpose:

- Call Groq model.
- Generate AI answer from source context.
- Support streaming.

Recommended model:

```
llama-3.3-70b-versatile
```

Fallback:

```
llama-3.1-8b-instant
```

Prompt rules:

```
You are an answer engine.
Use only provided sources.
Every factual claim needs citation like [1] or [2][3].
If sources do not contain the answer, say so.
Do not invent citation numbers.
```

4.6 `src/services/citation.service.js`

Purpose:

- Number sources.
- Create source cards.
- Validate model citations.
- Remove invalid citation numbers.

Why needed:

LLMs may return `[7]` even if only 5 sources exist. This service protects the UI from bad citations.

Functions it can expose:

```
numberSources(rawSources)
extractCitationIds(answer)
removeInvalidCitations(answer, sourceCount)
buildSourceCards(sources)
```

4.7 `src/services/stream.service.js`

Purpose:

- Central helper for SSE response headers and event formatting.

SSE headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

Event examples:

```
event: sources
data: [{"id":1,"title":"..."}]

event: token
data: {"text":"RAG"}

event: done
data: {"chatId":"..."}
```

4.8 `src/controllers/chat.controller.js`

Purpose:

- Manage saved chats.
- Return chat history.
- Return one chat thread.
- Rename/star/archive/delete chats.
- Add follow-up messages.

Needed endpoints:

```
GET    /api/chats
GET    /api/chats/:chatId
POST   /api/chats
POST   /api/chats/:chatId/messages
PATCH  /api/chats/:chatId
DELETE /api/chats/:chatId
```

4.9 `src/models/source.model.js`

Purpose:

- Store sources used in an answer.

Possible fields:

```
{
  chat: ObjectId,
  message: ObjectId,
  title: String,
  url: String,
  domain: String,
  snippet: String,
  favicon: String,
  citationId: Number,
  score: Number
}
```

4.10 `src/models/usage.model.js`

Purpose:

- Track usage limits for SaaS plans.

Possible fields:

```
{
  user: ObjectId,
  date: String,
  searchesUsed: Number,
  messagesUsed: Number,
```

```
  tokensUsed: Number,
  plan: String
}
```

4.11 `src/middlewares/rateLimit.middleware.js`

Purpose:

- Prevent API abuse.
- Limit unauthenticated and free users.

Example limits:

```
guest: 5 searches/hour
free: 20 searches/day
pro: 500 searches/day
```

4.12 `src/services/cache.service.js`

Purpose:

- Cache search results and scraped content.
- Reduce cost and improve speed.

Use Redis later.

Cache examples:

```
search:{hash(query)} -> 1 hour
scrape:{hash(url)} -> 6 hours
answer:{hash(query+sources)} -> 24 hours
```

5. Backend Features

5.1 Auth Feature

Already mostly implemented.

Responsibilities:

- Register user.
- Login user.
- Logout user.
- Verify email.
- Forgot password.
- Reset password.
- Return current user.

Main files:

```
auth.routes.js
auth.controller.js
auth.validator.js
auth.middleware.js
user.model.js
mail.service.js
```

Workflow:

```
Register form
  |
  v
POST /api/auth/register
  |
  v
Create user -> hash password -> send verification email
  |
  v
User verifies email
  |
  v
Login allowed
```

5.2 Search Feature

Not implemented yet.

Responsibilities:

- Run live web search.
- Retrieve sources.
- Scrape content.
- Create answer context.
- Generate cited answer.

Main files needed:

```
search.routes.js
search.controller.js
search.validator.js
search.service.js
scraper.service.js
groq.service.js
citation.service.js
queryRewrite.service.js
```

5.3 Streaming Feature

Not implemented yet.

Responsibilities:

- Stream answer tokens.
- Stream sources early if possible.
- Notify frontend when complete.
- Handle stream errors.

Frontend should show:

- Live typing.
- Blinking cursor.
- Source cards.
- Stop button later.

5.4 Chat History Feature

Partly modeled, not fully implemented.

Responsibilities:

- Save user message.
- Save AI answer.
- Save sources.
- Load chat by ID.
- Continue follow-up question.
- Rename/star/archive/delete chat.

Files needed:

```
chat.routes.js
chat.controller.js
chat.service.js
chat.validator.js
source.model.js
```

5.5 Library Feature

Backend not implemented yet.

The library is basically a query layer on chats.

Endpoints:

```
GET /api/library?filter=all
GET /api/library?filter=starred
GET /api/library?filter=shared
GET /api/library?search=rag
```

5.6 Settings Feature

Backend not implemented yet.

Responsibilities:

- Update profile.

- Update model preference.
- Update theme preference.
- Update privacy preference.
- Delete all chats.
- Delete account.

5.7 Billing Feature

Not implemented yet.

Responsibilities:

- Store plan.
- Track subscription status.
- Process payments.
- Enforce plan limits.

Possible payment APIs:

- Stripe.
- Razorpay.

Files:

```
billing.routes.js
billing.controller.js
billing.service.js
subscription.model.js
usage.model.js
subscription.middleware.js
```

5.8 File Upload Feature

Not implemented yet.

Claude-like feature.

Responsibilities:

- Upload PDF/image/doc.
- Extract text.
- Attach file to chat.
- Ask questions from file content.

Storage options:

- Local disk for development.
- Cloudinary/S3 for production.

Files:

```
file.routes.js
file.controller.js
file.service.js
```

```
file.model.js
upload.middleware.js
```

5.9 Spaces Feature

Not implemented yet.

Perplexity-like workspace feature.

Responsibilities:

- Create workspace.
- Add members.
- Save shared chats.
- Shared research context.

Files:

```
space.routes.js
space.controller.js
space.service.js
space.model.js
role.middleware.js
```

5.10 Public Sharing Feature

Not implemented yet.

Responsibilities:

- Create public read-only answer page.
- Allow anyone with link to view.
- Hide private data.

Files:

```
share.routes.js
share.controller.js
share.service.js
sharedPage.model.js
```

6. Backend API Plan

6.1 Auth APIs

Already mostly present:

```
POST /api/auth/register
POST /api/auth/login
GET  /api/auth/verify-email
POST /api/auth/forgot-password
```

```
POST /api/auth/reset-password
POST /api/auth/resend-verification
GET  /api/auth/me
POST /api/auth/logout
```

6.2 Search APIs

Need to create:

```
POST /api/search
POST /api/search/stream
```

Request body:

```
{
  "query": "What is RAG in AI?",
  "mode": "web",
  "model": "llama-3.3-70b-versatile",
  "chatId": null
}
```

Response:

```
{
  "chatId": "mongo-id",
  "answer": "RAG combines retrieval and generation [1][2].",
  "sources": [
    {
      "id": 1,
      "title": "Retrieval Augmented Generation",
      "url": "https://example.com",
      "domain": "example.com",
      "snippet": "RAG is..."
    }
  ]
}
```

6.3 Chat APIs

Need to create:

```
GET    /api/chats
GET    /api/chats/:chatId
POST   /api/chats/:chatId/messages
PATCH  /api/chats/:chatId
DELETE /api/chats/:chatId
```

6.4 Library APIs

Need to create:

```
GET /api/library
GET /api/library/starred
GET /api/library/shared
GET /api/library/archived
```

6.5 Settings/User APIs

Need to create:

```
GET    /api/users/me
PATCH  /api/users/me
PATCH  /api/settings/preferences
DELETE /api/users/me/chats
DELETE /api/users/me
```

6.6 Billing APIs

Need later:

```
GET  /api/billing/plans
POST /api/billing/checkout
POST /api/billing/webhook
GET  /api/billing/subscription
POST /api/billing/cancel
```

6.7 File APIs

Need later:

```
POST   /api/files
GET    /api/files
GET    /api/files/:fileId
DELETE /api/files/:fileId
```

6.8 Spaces APIs

Need later:

```
POST   /api/spaces
GET    /api/spaces
GET    /api/spaces/:spaceId
PATCH  /api/spaces/:spaceId
DELETE /api/spaces/:spaceId
POST   /api/spaces/:spaceId/members
DELETE /api/spaces/:spaceId/members/:userId
```

7. APIs And External Services Needed

7.1 LLM APIs

Recommended:

- Groq for fast and cheap LLM responses.
- OpenAI for mainstream reliability.
- Anthropic for Claude-like quality.

Start with:

```
Groq
Model: llama-3.3-70b-versatile
```

Environment variables:

```
GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile
```

7.2 Search APIs

Recommended:

- Tavily for AI search.
- Brave Search as backup.
- Exa for research/semantic search.

Start with:

```
TAVILY_API_KEY=
```

7.3 Scraping APIs

Recommended:

- Jina Reader for simple extraction.
- Firecrawl as stronger fallback.

Jina Reader pattern:

```
https://r.jina.ai/http://example.com
https://r.jina.ai/http://https://example.com
```

7.4 Database

Already using:

```
MongoDB + Mongoose
```

Need collections:

- users
- chats
- messages
- sources
- usage
- subscriptions
- files
- spaces
- sharedPages

7.5 Cache/Rate Limit

Later:

```
Redis
```

Used for:

- Search result cache.
- Scraped content cache.
- Rate limiting.
- Session/usage counters.

7.6 Email

Already has mail service.

Used for:

- Verify email.
- Forgot password.
- Billing emails later.

7.7 Billing

Later:

- Stripe for global.
- Razorpay for India.

7.8 File Storage

Later:

- Cloudinary.
- AWS S3.
- Local disk for development.

8. Frontend Architecture

8.1 Frontend Target Folder Structure

Recommended final structure:

```
Frontend/src/
  main.jsx

  app/
    App.jsx
    app.routes.jsx
    app.store.js
    index.css

  components/
    layout/
      AppLayout.jsx
      Sidebar.jsx
      Topbar.jsx
      UserMenu.jsx
      MobileNav.jsx

    ui/
      Button.jsx
      Input.jsx
      Textarea.jsx
      Modal.jsx
      Tabs.jsx
      Dropdown.jsx
      EmptyState.jsx
      Spinner.jsx
      Skeleton.jsx
      Tooltip.jsx
      Badge.jsx
      Toast.jsx

  data/
    promptSuggestions.js
    models.js
    searchModes.js
    plans.js

  lib/
    axios.js
    constants.js
    citationParser.js
    markdown.js
    formatDate.js
    streamParser.js
```

```
  features/
    auth/
    chat/
    search/
    library/
    settings/
    user/
    billing/
    files/
    spaces/
    share/
```

8.2 Frontend App Workflow

```
User opens app
  |
  v
Protected route checks auth
  |
  +-> not logged in -> Login/Register
  |
  +-> logged in -> AppLayout
                    |
                    +-> Sidebar: chat history
                    +-> Topbar: nav/settings/user
                    +-> Page content
```

8.3 Search/Chat Frontend Workflow

```
Home.jsx
  |
  | User enters query
  v
useChat.js
  |
  | sendMessage(query)
  v
chat.api.js
  |
  | POST /api/search or /api/search/stream
  v
Backend
  |
  | sources event
  | token events
  | done event
  v
useChat.js updates state
  |
```

```
  +-> MessageBubble renders user query
  +-> StreamingAnswer renders AI text
  +-> CitationBadge renders [1], [2]
  +-> SourcePanel renders source cards
```

8.4 Frontend Diagram

```
                 +--------------------+
                 |      App.jsx       |
                 +---------+----------+
                           |
                           v
                 +--------------------+
                 |   app.routes.jsx   |
                 +---------+----------+
                           |
              +-------------+-------------+
              |                           |
              v                           v
       Auth pages                  Protected app
              |                           |
              v                           v
     Login/Register              AppLayout.jsx
                                          |
                +-------------------------+-------------------------+
                |                         |                         |
                v                         v                         v
         Sidebar.jsx                Topbar.jsx                Page content
                |                                                   |
                v                                                   v
        Chat history
        Home/ChatThread/Library/Settings
```

Search page component flow:

```
Home.jsx
  |
  +-> ChatInput.jsx
  +-> SuggestedPrompts.jsx
  +-> ModelSelector.jsx
  +-> WebToggle.jsx

ChatThread.jsx
  |
  +-> ChatHeader.jsx
  +-> MessageBubble.jsx
  +-> StreamingAnswer.jsx
  +-> CitationBadge.jsx
  +-> SourcePanel.jsx
  +-> ChatInput.jsx
```

8.5 Frontend Files Explained

`src/app/App.jsx`

Purpose:

- Root React component.
- Renders router provider.

`src/app/app.routes.jsx`

Purpose:

- Defines frontend routes.

Needed routes:

```
/login
/register
/
/chat/:chatId
/library
/settings
/billing
/files
/spaces
/share/:shareId
```

`src/app/app.store.js`

Purpose:

- Redux store.
- Registers feature slices.

Should include:

```
auth: authReducer
chat: chatReducer
library: libraryReducer
settings: settingsReducer
user: userReducer
billing: billingReducer
```

`src/components/layout/AppLayout.jsx`

Purpose:

- Shared app shell.
- Contains sidebar, topbar, and page outlet.

Used by:

- Home.
- ChatThread.
- Library.
- Settings.
- Billing.

`src/components/layout/Sidebar.jsx`

Purpose:

- Shows New Chat button.
- Shows recent chats.
- Shows spaces.
- Shows bottom user area.

`src/components/layout/Topbar.jsx`

Purpose:

- Shows product name.
- Shows navigation.
- Shows settings/user buttons.

`src/features/chat/pages/Home.jsx`

Purpose:

- Main starting page.
- Big search input.
- Suggested prompts.
- Model selector.
- Search mode selector.

`src/features/chat/pages/ChatThread.jsx`

Purpose:

- Displays a full conversation.
- Shows user messages.
- Shows AI answers.
- Shows source panel.
- Shows follow-up input.

`src/features/chat/components/ChatInput.jsx`

Purpose:

- Reusable input for initial query and follow-up questions.

Needs:

- Textarea.
- Submit button.

- Enter to submit.
- Shift+Enter for newline.
- Loading disabled state.

`src/features/chat/components/MessageBubble.jsx`

Purpose:

- Renders one user or AI message.

For AI messages:

- Use markdown.
- Parse citations.
- Show copy/regenerate actions.

`src/features/chat/components/SourcePanel.jsx`

Purpose:

- Shows source cards.
- On desktop, right side.
- On mobile, below answer.

Source card fields:

```
id
title
domain
url
snippet
favicon
score
```

`src/features/chat/components/CitationBadge.jsx`

Not created yet but should be added.

Purpose:

- Renders `[1]` as a clickable pill.
- On hover, shows source preview.
- On click, scrolls/highlights source card.

`src/features/chat/components/StreamingAnswer.jsx`

Not created yet but should be added.

Purpose:

- Renders answer while tokens are arriving.
- Shows blinking cursor.
- Uses MarkdownRenderer.

`src/features/library/pages/Library.jsx`

Purpose:

- Shows all saved chats.
- Supports filters:
  - all
  - starred
  - shared
  - archived
  - code chats

`src/features/settings/pages/Settings.jsx`

Purpose:

- Profile.
- Security.
- Notifications.
- Billing.
- Models.
- API keys.
- Memory.
- Appearance.
- Privacy.
- Data export.

9. Frontend Design And UI/UX

9.1 Design Direction

The UI should feel like a serious SaaS workspace:

- Dense but clean.
- Fast to scan.
- Quiet visual design.
- No marketing landing page as the main app screen.
- Search/chat is the first experience.
- Source trust is visually important.

9.2 Color Theory

Recommended dark theme:

```
Page background:   #0F1117
Primary surface:   #161A22
Card surface:      #1A1D27
Input surface:     #202634
Border:            #2B3241
Primary text:      #F3F5F7
Secondary text:    #A7ADB8
```

```
Muted text:        #737B8C
Accent:            #20B2AA
Accent hover:      #37C9C0
Danger:            #EF4444
Warning:           #F59E0B
Success:           #22C55E
```

Why teal/cyan:

- It feels technical and trustworthy.
- It avoids the overused purple AI-app look.
- It works well on dark backgrounds.
- It can signal source/citation interactivity clearly.

9.3 Typography

Recommended:

```
UI font: Inter or Geist
Code font: Geist Mono or JetBrains Mono
```

Use:

- 14-16px for body UI.
- 13px for secondary metadata.
- 12px for labels/badges.
- 20-28px for page titles only.

Avoid:

- Huge marketing headings inside the app.
- Negative letter spacing.
- Viewport-based font sizes.

9.4 Layout Rules

Desktop chat page:

```
+----------------+-------------------------------+------------------+
| Sidebar 240px  | Answer/content area           | Sources 320-360px |
+----------------+-------------------------------+------------------+
```

Mobile chat page:

```
Topbar
Search/answer
Sources below answer
Bottom input
```

9.5 Home Page Design

The home page should focus on one action:

```
Ask a question
```

Should include:

- Centered search box.
- Suggested prompts.
- Model selector.
- Web toggle.
- Recent chats in sidebar.

Should not include:

- Marketing hero.
- Big decorative gradients.
- Too many cards.

9.6 Chat Thread Design

Main sections:

- Left sidebar.
- Chat title/header.
- Message stream.
- Source panel.
- Follow-up input.

AI answer must support:

- Markdown.
- Bullets.
- Code blocks.
- Tables later.
- Citations.
- Copy/regenerate actions.

9.7 Source Panel Design

Source cards should show:

```
[1] domain.com
Title
One-line snippet
```

Avoid showing huge paragraphs. The source panel should be scannable.

9.8 Loading States

Use skeletons matching final shapes:

- Source card skeletons.
- Answer line skeleton or streaming cursor.
- Chat list skeleton.

Avoid a generic spinner as the only loading state.

9.9 Error States

Examples:

- Search API failed.
- Scraping failed.
- LLM timed out.
- Usage limit reached.
- User not logged in.

UI should show clear recovery:

- Retry.
- Use snippets only.
- Choose another model.
- Upgrade plan.

9.10 Accessibility

Needed:

- Keyboard submit.
- Visible focus rings.
- Proper labels.
- Buttons have accessible names.
- Source cards are reachable by keyboard.
- Color contrast on dark theme.

10. Frontend Features

10.1 Auth

Already implemented mostly.

Needs final polish:

- Ensure error messages are clear.
- Ensure `/me` check on reload.
- Redirect logged-in users away from login/register.

10.2 Chat/Search

Needs implementation.

State needed:

```
{
  currentChatId,
  messages,
```

```
  sources,
  loading,
  streaming,
  error,
  selectedModel,
  searchMode,
  webEnabled
}
```

10.3 Library

Needs implementation.

Data needed:

```
{
  chats,
  filter,
  searchQuery,
  loading,
  error
}
```

10.4 Settings

Needs implementation.

Data needed:

```
{
  profile,
  preferences,
  plan,
  loading,
  error
}
```

10.5 Billing

Not created yet.

Frontend screens:

- Billing page.
- Plan cards.
- Usage meter.
- Checkout button.
- Current subscription status.

10.6 Files

Not created yet.

Frontend screens:

- Files page.
- Upload dropzone.
- File list.
- File-attached chat state.

10.7 Spaces

Not created yet.

Frontend screens:

- Spaces list.
- Space detail page.
- Member list.
- Shared chat list.

10.8 Share

Not created yet.

Frontend screens:

- Public answer page.
- Read-only source panel.
- Share link modal.

11. What To Build First

Because you want a real SaaS project, still build in a controlled order.

Phase 1: Core Answer Engine

Backend:

```
search.routes.js
search.controller.js
search.validator.js
search.service.js
scraper.service.js
groq.service.js
citation.service.js
```

Frontend:

```
lib/axios.js
features/chat/service/chat.api.js
features/chat/hook/useChat.js
features/chat/pages/Home.jsx
features/chat/pages/ChatThread.jsx
features/chat/components/ChatInput.jsx
features/chat/components/MessageBubble.jsx
features/chat/components/SourcePanel.jsx
features/chat/components/CitationBadge.jsx
features/chat/components/StreamingAnswer.jsx
```

Goal:

```
Ask question -> get cited answer -> see source cards
```

Phase 2: Streaming

Backend:

```
stream.service.js
POST /api/search/stream
```

Frontend:

```
lib/streamParser.js
StreamingAnswer.jsx
```

Goal:

```
Answer appears token-by-token
```

Phase 3: Chat History

Backend:

```
chat.routes.js
chat.controller.js
chat.service.js
source.model.js
```

Frontend:

```
Sidebar.jsx
ChatSidebar.jsx
Library.jsx
ChatListItem.jsx
```

Goal:

```
Saved conversations and follow-up questions
```

Phase 4: SaaS Limits

Backend:

```
usage.model.js
usage.service.js
rateLimit.middleware.js
subscription.middleware.js
```

Frontend:

```
billing/
UsageMeter.jsx
```

Goal:

```
Free/pro limits
```

Phase 5: Advanced Features

Add:

- Billing.
- Files.
- Spaces.
- Sharing.
- Caching.
- Admin tools.

12. Done vs Remaining

12.1 Done

Backend:

- Express server.
- MongoDB connection.
- Auth route structure.
- Auth controller.
- JWT middleware.
- User model.
- Chat model.
- Message model.
- Mail service.
- Error handler.
- Validator middleware.

Frontend:

- React/Vite setup.
- Router setup.
- Redux setup.

- Auth slice.
- Login page.
- Register page.
- Protected component.
- Base folder structure.
- Placeholder files for layout, UI, chat, library, settings.

12.2 Remaining

Backend:

- Real search API integration.
- Real scraper integration.
- Real Groq/OpenAI/Claude service.
- Non-streamed answer endpoint.
- SSE streamed answer endpoint.
- Citation validation.
- Source model.
- Chat route/controller/service.
- Library APIs.
- Settings APIs.
- User profile APIs.
- Usage tracking.
- Rate limiting.
- Billing.
- File upload.
- Spaces.
- Share pages.
- Redis cache.

Frontend:

- Fill empty layout components.
- Fill empty UI components.
- Build Home page.
- Build ChatThread page.
- Build chat input.
- Build streaming answer.
- Build citation badges.
- Build source cards/panel.
- Build markdown renderer.
- Build library page.
- Build settings page.
- Build billing page.
- Build files page.
- Build spaces page.
- Build share page.
- Connect all APIs.
- Add responsive design.
- Add loading/error/empty states.

13. Environment Variables Needed

Backend `.env` should eventually include:

```
PORT=3000
MONGODB_URI=
JWT_SECRET=
JWT_EXPIRES_IN=

FRONTEND_URL=http://localhost:5173

GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile

TAVILY_API_KEY=
BRAVE_SEARCH_API_KEY=
EXA_API_KEY=

JINA_READER_BASE_URL=https://r.jina.ai/http://
FIRECRAWL_API_KEY=

REDIS_URL=

SMTP_HOST=
SMTP_PORT=
SMTP_USER=
SMTP_PASS=

STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=

CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

Frontend `.env` should eventually include:

```
VITE_API_BASE_URL=http://localhost:3000/api
VITE_APP_NAME=YourAI
```

14. Final Recommended Architecture

```
Browser
  |
  v
React/Vite frontend
  |
  +-> Auth pages
```

```
  +-> App layout
  +-> Chat/search UI
  +-> Library
  +-> Settings
  +-> Billing/files/spaces later
  |
  v
Express backend
  |
  +-> Auth APIs
  +-> Search APIs
  +-> Chat APIs
  +-> User/settings APIs
  +-> Billing/file/space/share APIs later
  |
  +-> MongoDB
  +-> Redis later
  +-> Groq/OpenAI/Anthropic
  +-> Tavily/Brave/Exa
  +-> Jina/Firecrawl
  +-> Stripe/Razorpay later
```

The core success of the project depends on this one loop:

```
query -> search -> scrape -> synthesize -> cite -> stream -> save -> continue
```

Everything else is SaaS packaging around that loop.